

# Rajshahi University of Engineering & Technology



Department of Electrical and Computer Engineering

---

## Lab - 04: Numerical Estimation Using Linear, Lagrange, and Newton Forward Interpolation

---

**Course Title:** Numerical Methods & Discrete Mathematics  
Sessional

**Course Code:** ECE 2214

*Submitted by:*  
**Md. Yousuf**  
Roll: 2310023

*Submitted to:*  
**Nadim Reza**  
Lecturer  
Dept. of ECE, RUET

# Contents

|          |                                                   |          |
|----------|---------------------------------------------------|----------|
| <b>1</b> | <b>Name of the Experiment</b>                     | <b>1</b> |
| <b>2</b> | <b>Theory</b>                                     | <b>1</b> |
| 2.1      | Linear Interpolation . . . . .                    | 1        |
| 2.2      | Lagrange Interpolation . . . . .                  | 1        |
| 2.3      | Newton Forward Difference Interpolation . . . . . | 1        |
| <b>3</b> | <b>Algorithm</b>                                  | <b>2</b> |
| <b>4</b> | <b>Source Code</b>                                | <b>3</b> |
| <b>5</b> | <b>Experimental Outputs</b>                       | <b>6</b> |
| 5.1      | Linear Interpolation Output . . . . .             | 6        |
| 5.2      | Lagrange Interpolation Output . . . . .           | 7        |
| 5.3      | Newton Forward Interpolation Output . . . . .     | 7        |
| <b>6</b> | <b>Code Explanation</b>                           | <b>8</b> |
| <b>7</b> | <b>Discussion</b>                                 | <b>9</b> |
| <b>8</b> | <b>References</b>                                 | <b>9</b> |

## List of Figures

|   |                                       |   |
|---|---------------------------------------|---|
| 1 | Console Output (Linear) . . . . .     | 6 |
| 2 | Plotted Graph (Linear) . . . . .      | 6 |
| 3 | Console Output (Lagrange) . . . . .   | 7 |
| 4 | Plotted Graph (Lagrange) . . . . .    | 7 |
| 5 | Console Output (Newton Fwd) . . . . . | 7 |
| 6 | Plotted Graph (Newton Fwd) . . . . .  | 7 |

## List of Tables

|   |                                                   |   |
|---|---------------------------------------------------|---|
| 1 | Structure of a Forward Difference Table . . . . . | 2 |
|---|---------------------------------------------------|---|

# 1 Name of the Experiment

Numerical Estimation Using Linear, Lagrange, and Newton Forward Interpolation.

## 2 Theory

Interpolation is a method of estimating unknown data points within the range of a discrete set of known data points. In engineering and science, one often has a number of data points, obtained by sampling or experimentation, which represent the values of a function for a limited number of values of the independent variable. This experiment explores three specific methods:

### 2.1 Linear Interpolation

Linear interpolation is the simplest form of interpolation, connecting two data points with a straight line. If the two known points are given by  $(x_0, y_0)$  and  $(x_1, y_1)$ , the linear interpolant is the straight line between these points. For a value  $x$  in the interval  $(x_0, x_1)$ , the value  $y$  is given by:

$$y = y_0 + (x - x_0) \frac{y_1 - y_0}{x_1 - x_0} \quad (1)$$

### 2.2 Lagrange Interpolation

Lagrange polynomials are used for polynomial interpolation. For a given set of points  $(x_j, y_j)$  with no two  $x_j$  values equal, the Lagrange polynomial is the linear combination:

$$P(x) = \sum_{j=0}^n y_j L_j(x) \quad (2)$$

Where  $L_j(x)$  are the Lagrange basis polynomials defined as:

$$L_j(x) = \prod_{\substack{0 \leq m \leq n \\ m \neq j}} \frac{x - x_m}{x_j - x_m} \quad (3)$$

### 2.3 Newton Forward Difference Interpolation

Newton's forward difference formula is highly effective when the data points are equally spaced. It utilizes a forward difference table. Let  $h = x_{i+1} - x_i$  be the step size, and  $p = \frac{x - x_0}{h}$ . The formula is given by:

$$f(x) = y_0 + p\Delta y_0 + \frac{p(p-1)}{2!}\Delta^2 y_0 + \frac{p(p-1)(p-2)}{3!}\Delta^3 y_0 + \dots \quad (4)$$

Table 1 demonstrates the conceptual structure of a forward difference table used in this method.

Table 1: Structure of a Forward Difference Table

| $x$   | $y = f(x)$ | $\Delta y$ (1st Diff)    | $\Delta^2 y$ (2nd Diff)                  | $\Delta^3 y$ (3rd Diff) |
|-------|------------|--------------------------|------------------------------------------|-------------------------|
| $x_0$ | $y_0$      | $\Delta y_0 = y_1 - y_0$ | $\Delta^2 y_0 = \Delta y_1 - \Delta y_0$ | $\Delta^3 y_0$          |
| $x_1$ | $y_1$      | $\Delta y_1 = y_2 - y_1$ | $\Delta^2 y_1 = \Delta y_2 - \Delta y_1$ |                         |
| $x_2$ | $y_2$      | $\Delta y_2 = y_3 - y_2$ |                                          |                         |
| $x_3$ | $y_3$      |                          |                                          |                         |

### 3 Algorithm

1. Start the program and clear the workspace and command window.
2. Prompt the user to input an anonymous function  $f(x)$ , a vector of known data points  $x$ , and a vector of target estimation points  $x_t$ .
3. Calculate the true  $y$  values corresponding to the input  $x$  array.
4. Present a menu displaying three interpolation methods: Linear, Lagrange, and Newton Forward. Capture the user's choice.
5. Use a switch-case statement to execute the chosen method, passing a boolean flag (`show_tab = true`) to generate detailed iteration tables:
  - **If Linear:** Find the interval  $[x_0, x_1]$  bounding the target point  $x_t$ , apply the linear formula, and print an iteration table showing the chosen coordinates and estimates.
  - **If Lagrange:** Calculate the basis polynomials for all points, accumulate the sum, and print a step-by-step iteration table for each  $x_t$  showing the basis multiplier and added terms.
  - **If Newton Forward:** Construct an  $n \times n$  difference matrix and print it. Calculate  $p$ , sum the terms using the forward difference formula, and print a step-by-step table showing each term added to the final estimate.
6. Print the final estimated values for the selected method.
7. Run all three methods again in the background with the table flag set to `false` to silently gather all numerical results.
8. Print a comparative matrix table of all method estimations.
9. Calculate and print the absolute errors by comparing the estimated values against the true analytical values.
10. Generate a linearly spaced set of 200 points ( $x_{smooth}$ ) to calculate a continuous mathematical curve for the specifically chosen method (again, with `show_tab = false`).

11. Plot the true function curve, the specific interpolated curve, the known data points, and the estimated points on a 2D graph.
12. End the program.

## 4 Source Code

```

1 function lab_04()
2     clc; close all;
3
4     f_true = input('Enter the function: ');
5     x = input('Enter the x data points as a vector: ');
6     xt = input('Enter the target values to estimate as a vector: ');
7
8     y = f_true(x);
9     yt = zeros(size(xt));
10    method_name = '';
11
12    fprintf('\n1. Linear\n2. Lagrange\n3. Newton Forward\n');
13    choice = input('Select a method (1-3): ');
14
15    fprintf('\n');
16    switch choice
17        case 1
18            method_name = 'Linear';
19            yt = calc_linear(x, y, xt, true);
20        case 2
21            method_name = 'Lagrange';
22            yt = calc_lagrange(x, y, xt, true);
23        case 3
24            method_name = 'Newton Forward';
25            [yt, D] = calc_newton_forward(x, y, xt, true);
26        otherwise
27            choice = 1;
28            method_name = 'Linear';
29            yt = calc_linear(x, y, xt, true);
30    end
31
32    fprintf('\nEstimated Values using %s:\n', method_name);
33    for i = 1:length(xt)
34        fprintf('f(%.4f) = %.4f\n', xt(i), yt(i));
35    end
36
37    res_lin = calc_linear(x, y, xt, false);
38    res_lag = calc_lagrange(x, y, xt, false);
39    [res_nf, ~] = calc_newton_forward(x, y, xt, false);
40
41    all_results = [res_lin; res_lag; res_nf];
42    method_names = {'Linear', 'Lagrange', 'Newton Fwd'};
43
44    fprintf('\nMethod\n');
45    for i = 1:length(xt)
46        fprintf('f(%.4f)\n', xt(i));
47    end
48    fprintf('\n');
49

```

```

50     for i = 1:3
51         fprintf('%-18s ', method_names{i});
52         for j = 1:length(xt)
53             fprintf('%-12.6f ', all_results(i, j));
54         end
55         fprintf('\n');
56     end
57
58     fprintf('\nDifference from True Analytical Value:\n');
59     true_vals = f_true(xt);
60     for i = 1:3
61         fprintf('%-18s ', method_names{i});
62         for j = 1:length(xt)
63             err = abs(all_results(i, j) - true_vals(j));
64             fprintf('Error at %.2f: %-10.6f | ', xt(j), err);
65         end
66         fprintf('\n');
67     end
68
69     figure('Name', 'Interpolation Results', 'NumberTitle', 'off');
70     x_smooth = linspace(min(x)-0.5, max(x)+0.5, 200);
71     y_true_smooth = f_true(x_smooth);
72
73     if choice == 2
74         y_interp_smooth = calc_lagrange(x, y, x_smooth, false);
75     elseif choice == 3
76         [y_interp_smooth, ~] = calc_newton_forward(x, y, x_smooth,
77 false);
78     else
79         y_interp_smooth = calc_linear(x, y, x_smooth, false);
80     end
81
82     plot(x_smooth, y_true_smooth, '--k', 'LineWidth', 1.2); hold on;
83     plot(x_smooth, y_interp_smooth, '-b', 'LineWidth', 1.5);
84     plot(x, y, 'ko', 'MarkerSize', 8, 'MarkerFaceColor', 'k');
85     plot(xt, yt, 'r*', 'MarkerSize', 10, 'LineWidth', 2);
86
87     title(sprintf('Interpolation using %s Method', method_name));
88     xlabel('x'); ylabel('f(x)');
89     legend('True Function', sprintf('%s Curve', method_name), 'Given
Data Points', 'Interpolated Points', 'Location', 'NorthWest');
90     grid on; hold off;
91 end
92
93 function y_est = calc_linear(x, y, xt, show_tab)
94     y_est = zeros(size(xt));
95     if show_tab
96         fprintf('--- Linear Iteration Table ---\n');
97         fprintf('%-10s %-10s %-10s %-10s %-10s %-10s\n', 'xt', 'x0', '
x1', 'y0', 'y1', 'Estimate');
98     end
99     for k = 1:length(xt)
100         idx = find(x <= xt(k), 1, 'last');
101         if isempty(idx) || idx == length(x), idx = length(x) - 1; end
102         x0 = x(idx); x1 = x(idx+1);
103         y0 = y(idx); y1 = y(idx+1);
104         y_est(k) = y0 + (xt(k) - x0) * (y1 - y0) / (x1 - x0);
105         if show_tab

```

```

105         fprintf('%-10.4f %-10.4f %-10.4f %-10.4f %-10.4f %-10.4f\n',
, xt(k), x0, x1, y0, y1, y_est(k));
106     end
107 end
108 end
109
110 function y_est = calc_lagrange(x, y, xt, show_tab)
111     n = length(x);
112     y_est = zeros(size(xt));
113     for k = 1:length(xt)
114         sum = 0;
115         if show_tab
116             fprintf('\n--- Lagrange Iteration for xt = %.4f ---\n', xt(
k));
117             fprintf('%-5s %-10s %-10s %-10s %-10s %-10s\n', 'i', 'x(i)',
, 'y(i)', 'L_i', 'Term', 'Sum');
118             end
119             for i = 1:n
120                 product = 1;
121                 for j = 1:n
122                     if i ~= j
123                         product = product * (xt(k) - x(j)) / (x(i) - x(j));
124                     end
125                 end
126                 term = y(i) * product;
127                 sum = sum + term;
128                 if show_tab
129                     fprintf('%-5d %-10.4f %-10.4f %-10.4f %-10.4f %-10.4f\n
, i, x(i), y(i), product, term, sum);
130                 end
131             end
132             y_est(k) = sum;
133         end
134     end
135
136 function [y_est, D] = calc_newton_forward(x, y, xt, show_tab)
137     n = length(x);
138     D = zeros(n, n); D(:,1) = y(:);
139     for j = 2:n
140         for i = 1:n-j+1
141             D(i,j) = D(i+1,j-1) - D(i,j-1);
142         end
143     end
144     if show_tab
145         fprintf('--- Newton Forward Difference Table ---\n');
146         disp(D);
147     end
148     h = x(2) - x(1);
149     y_est = zeros(size(xt));
150     for k = 1:length(xt)
151         p = (xt(k) - x(1)) / h;
152         val = D(1,1); term = 1;
153         if show_tab
154             fprintf('\n--- Newton Fwd Iteration for xt = %.4f (p = %.4f
) ---\n', xt(k), p);
155             fprintf('%-5s %-12s %-12s %-12s %-12s\n', 'j', 'Multiplier'
, 'D(1,j)', 'Added Term', 'Estimate');
156             fprintf('%-5d %-12.4f %-12.4f %-12.4f %-12.4f\n', 1,

```

```

1.0000, D(1,1), D(1,1), val);
157     end
158     for j = 2:n
159         term = term * (p - j + 2) / (j - 1);
160         added_term = term * D(1,j);
161         val = val + added_term;
162         if show_tab
163             fprintf('%-5d %-12.4f %-12.4f %-12.4f %-12.4f\n', j,
164                 term, D(1,j), added_term, val);
165         end
166         y_est(k) = val;
167     end
168 end

```

Listing 1: Interpolation Methods Implementation with Iteration Tables

## 5 Experimental Outputs

The experimental code was tested using the following dataset:

- **Function:**  $f(x) = e^{0.5x} \sin(x)$
- **Given Data Points (x):** [0, 1, 2, 3, 4, 5]
- **Target Values ( $x_t$ ):** [1.5, 3.8]

### 5.1 Linear Interpolation Output

```

Enter the function: @(x) exp(0.5 .* x) .* sin(x)
Enter the x data points as a vector: [0, 1, 2, 3, 4, 5]
Enter the target values to estimate as a vector: [1.5, 3.8]

1. Linear
2. Lagrange
3. Newton Forward
Select a method (1-3): 1

--- Linear Iteration Table ---
xt      x0      x1      y0      y1      Estimate
1.5000   1.0000   2.0000   1.3874   2.4717   1.9295
3.8000   3.0000   4.0000   0.6325  -5.5921  -4.3472

Estimated Values using Linear:
f(1.5000) = 1.9295
f(3.8000) = -4.3472

Method      f(1.5000)      f(3.8000)
Linear      1.929539      -4.347154
Lagrange    2.101858      -4.088263
Newton Fwd  2.101858      -4.088263

Difference from True Analytical Value:
Linear      Error at 1.50: 0.182158 | Error at 3.80: 0.256336 |
Lagrange    Error at 1.50: 0.009839 | Error at 3.80: 0.002554 |
Newton Fwd  Error at 1.50: 0.009839 | Error at 3.80: 0.002554 |

```

Figure 1: Console Output (Linear)

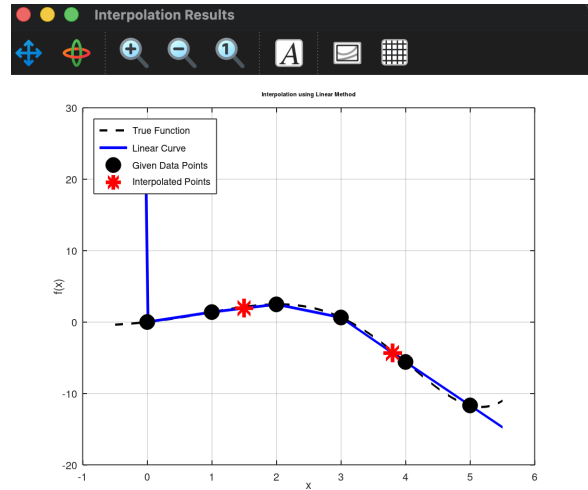


Figure 2: Plotted Graph (Linear)

## 5.2 Lagrange Interpolation Output

Enter the function: @(x) exp(0.5 .\* x) .\* sin(x)  
Enter the x data points as a vector: [0, 1, 2, 3, 4, 5]  
Enter the target values to estimate as a vector: [1.5, 3.8]

1. Linear  
2. Lagrange  
3. Newton Forward  
Select a method (1-3): 2

```
--- Lagrange Iteration for xt = 1.5000 ---
i  x(i)  y(i)  L_i  Term  Sum
1  0.0000  0.0000  -0.0273  -0.0000  0.0000
2  1.0000  1.3874  0.4102  0.5690  0.5690
3  2.0000  2.4717  0.8203  2.0276  2.5966
4  3.0000  0.6325  -0.2734  -0.1729  2.4237
5  4.0000  -5.5921  0.0820  -0.4587  1.9650
6  5.0000  -11.6821  -0.0117  0.1369  2.1019
```

```
--- Lagrange Iteration for xt = 3.8000 ---
i  x(i)  y(i)  L_i  Term  Sum
1  0.0000  0.0000  -0.0081  -0.0000  0.0000
2  1.0000  1.3874  0.0547  0.0759  0.0759
3  2.0000  2.4717  -0.1702  -0.4208  -0.3449
4  3.0000  0.6325  0.3830  0.2423  -0.1026
5  4.0000  -5.5921  0.7661  -4.2840  -4.3866
6  5.0000  -11.6821  -0.0255  0.2983  -4.0883
```

Estimated Values using Lagrange:  
 $f(1.5000) = 2.1019$   
 $f(3.8000) = -4.0883$

| Method     | $f(1.5000)$ | $f(3.8000)$ |
|------------|-------------|-------------|
| Linear     | 1.929539    | -4.347154   |
| Lagrange   | 2.101858    | -4.088263   |
| Newton Fwd | 2.101858    | -4.088263   |

|            | Error at 1.50: | Error at 3.80: |
|------------|----------------|----------------|
| Linear     | 0.182158       | 0.256336       |
| Lagrange   | 0.009839       | 0.002554       |
| Newton Fwd | 0.009839       | 0.002554       |

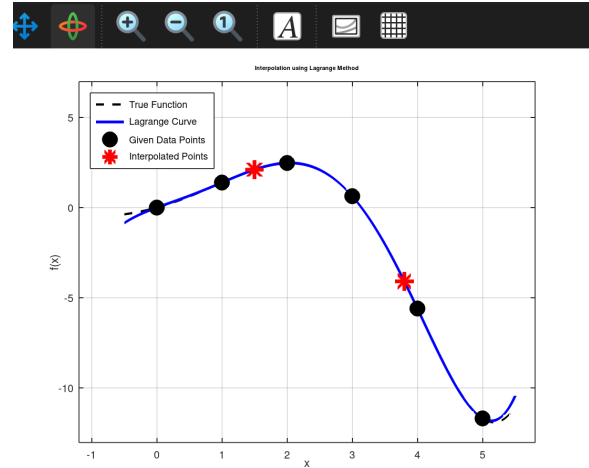


Figure 4: Plotted Graph (Lagrange)

Figure 3: Console Output (Lagrange)

## 5.3 Newton Forward Interpolation Output

Enter the function: @(x) exp(0.5 .\* x) .\* sin(x)  
Enter the x data points as a vector: [0, 1, 2, 3, 4, 5]  
Enter the target values to estimate as a vector: [1.5, 3.8]

1. Linear  
2. Lagrange  
3. Newton Forward  
Select a method (1-3): 3

```
--- Newton Forward Difference Table ---
      0      1.3874  -0.3030  -2.6207  1.1591  4.8222
    1.3874  1.0844  -2.9236  -1.4616  5.9813  0
    2.4717  -1.8393  -4.3852  4.5197  0
    0.6325  -6.2245  0.1345  0
   -5.5921  -6.0900  0
  -11.6821  0  0  0  0
```

```
--- Newton Fwd Iteration for xt = 1.5000 (p = 1.5000) ---
j  Multiplier  D(1,j)  Added Term  Estimate
1  1.0000  0.0000  0.0000  0.0000
2  1.5000  1.3874  2.0810  2.0810
3  0.3750  -0.3030  -0.1136  1.9674
4  -0.0625  -2.6207  0.1638  2.1312
5  0.0234  1.1591  0.0272  2.1584
6  -0.0117  4.8222  -0.0565  2.1019
```

```
--- Newton Fwd Iteration for xt = 3.8000 (p = 3.8000) ---
j  Multiplier  D(1,j)  Added Term  Estimate
1  1.0000  0.0000  0.0000  0.0000
2  3.8000  1.3874  5.2719  5.2719
3  5.3200  -0.3030  -1.6118  3.6601
4  3.1920  -2.6207  -8.3652  -4.7051
5  0.6384  1.1591  0.7400  -3.9651
6  -0.0255  4.8222  -0.1231  -4.0883
```

Estimated Values using Newton Forward:  
 $f(1.5000) = 2.1019$   
 $f(3.8000) = -4.0883$

| Method     | $f(1.5000)$ | $f(3.8000)$ |
|------------|-------------|-------------|
| Linear     | 1.929539    | -4.347154   |
| Lagrange   | 2.101858    | -4.088263   |
| Newton Fwd | 2.101858    | -4.088263   |

|            | Error at 1.50: | Error at 3.80: |
|------------|----------------|----------------|
| Linear     | 0.182158       | 0.256336       |
| Lagrange   | 0.009839       | 0.002554       |
| Newton Fwd | 0.009839       | 0.002554       |

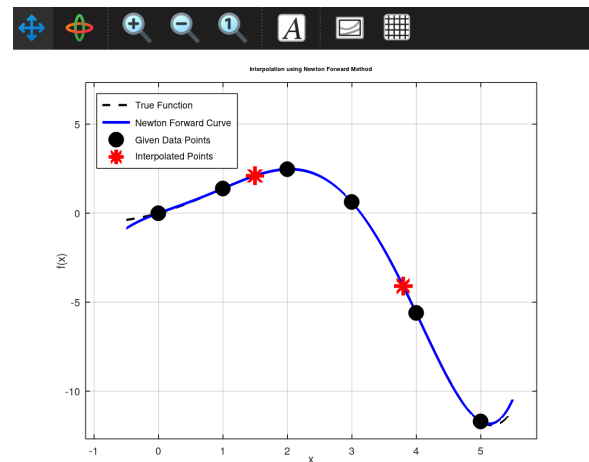


Figure 6: Plotted Graph (Newton Fwd)

Figure 5: Console Output (Newton Fwd)

## 6 Code Explanation

The provided MATLAB/Octave script is structured to handle inputs, display highly detailed iteration tables, compute method comparisons, and output graphical plots. Below is a detailed breakdown of the code logic:

### 1. Initialization and Method Selection

- `clc; close all;`: Clears the console and closes existing plot windows.
- `input(...)`: Dynamically prompts the user for the mathematical function, known data points ( $x$ ), and target values ( $x_t$ ).
- A `switch-case` block routes the execution based on the user's choice. Notably, the selected function is called with a boolean parameter `true` (e.g., `calc_lagrange(x, y, xt, true)`). This acts as a flag telling the function to explicitly print its internal step-by-step iteration table to the console.

### 2. Background Computation and Error Analysis

- To display the differences between all methods at the end, the script recalculates all three methods in the background. During this step, the functions are called with `false` (e.g., `calc_linear(x, y, xt, false)`). This suppresses the iteration table printouts so the console does not become cluttered with redundant tables.
- The final outputs are compiled into a matrix (`all_results`), and a formatted summary table is printed showing the final estimates side-by-side.
- `err = abs(all_results(i, j) - true_vals(j));`: Subtracts the estimated values from the true analytical values to determine the absolute error margins.

### 3. Graphical Plotting

- Plotting commands are placed at the end of the script to avoid macOS UI server terminal glitches.
- `x_smooth = linspace(...)`; generates 200 fine data points to draw continuous curves.
- The chosen interpolation function is called one final time (with `show_tab = false`) to map out the exact polynomial curve of the method. The `plot(...)` commands then overlay the true function, the interpolated function, the known points, and the estimated targets.

### 4. Function Definitions & Iteration Tables

- `calc_linear`: Iterates through the target points and uses `find()` to identify adjacent left/right values. If `show_tab` is true, it outputs a table row showing  $x_0$ ,  $x_1$ ,  $y_0$ ,  $y_1$ , and the resulting estimate for each target point.

- **calc\_lagrange**: Uses nested loops. The inner loop calculates the basis product ( $L_i$ ), and the outer loop scales it by  $y_i$ . If **show\_tab** is true, it prints a highly detailed table showing the current index  $i$ , the basis multiplier, the term added in that step, and the running sum.
- **calc\_newton\_forward**: First, builds the Forward Difference Matrix ( $D$ ) and prints it. It computes the step multiplier  $p = (x_t - x_1)/h$ . As it sums the top-diagonal elements across the loop, it prints an iteration table showing the specific  $p$ -multiplier, the selected difference matrix value, the newly calculated term, and the running total estimate.

## 7 Discussion

In this experiment, three numerical interpolation techniques were implemented and tested against an exponential sine wave function,  $f(x) = e^{0.5x} \sin(x)$ . A major feature of this updated algorithm was the generation of internal iteration tables, allowing for a transparent view into how each mathematical method converges on its final estimate.

From the graphical plots and console error tables, it is evident that Linear interpolation produces the highest error, as it merely connects adjacent data points with rigid, straight lines. The linear iteration table confirmed that it only utilizes the two immediately bounding data points, entirely ignoring the broader curvature of the trigonometric function.

Conversely, Lagrange and Newton Forward interpolation fit a single, continuous polynomial through all available data points. Because the test function is smooth, these higher-degree polynomials tracked the true mathematical curve much more accurately, yielding significantly lower error values at the target estimation points ( $x_t = 1.5$  and  $3.8$ ).

By analyzing the newly implemented step-by-step iteration tables, the structural differences between Lagrange and Newton Forward became very clear, despite them mathematically arriving at the exact same final estimation. The Lagrange table demonstrated how each individual data point contributes a weighted "term" to the final sum via its basis polynomial. In contrast, the Newton Forward table demonstrated a systematic progression: starting with a base  $y$ -value and incrementally refining the estimate by adding higher-order difference terms (scaled by the  $p$ -multiplier) derived from its pre-calculated difference matrix.

## 8 References

1. Chapra, S. C., & Canale, R. P. (2015). *Numerical Methods for Engineers*. McGraw-Hill Education.
2. Burden, R. L., & Faires, J. D. (2010). *Numerical Analysis*. Cengage Learning.
3. MathWorks Documentation. (n.d.). *Interpolation*. Retrieved from <https://www.mathworks.com/help/matlab/interpolation.html>